Failure and Recovery Drills

Order Processing API: what breaks when a component stops, slows down, or delivers the same work twice

Run date	28 Oct 2026, 05:27 to 05:55 UTC (10:57 to 11:25 IST)
Code	branch `feature/load-testing` at 446347a (working tree clean), API built from it as `bin/server.exe`
Environment	Local only. Windows 11 host running the API; Docker Desktop (WSL2) running postgres:16 (port 5434), Kafka 3.2 KRaft, Debezium Connect 3.2, OTel/Jaeger/Prometheus/Grafana from docker-compose.yml. No hosted (Render/Neon/Netlify) resource and no Razorpay endpoint was touched.
Load	Python drill harness: 4 to 300 threads placing one-unit orders as the seeded `loadtest@example.com` customer on the LOADTEST-0001..0010 fixtures, every attempt recorded with status and latency.
Checks after every drill	(1) every order acknowledged with 201 exists in PostgreSQL; (2) orders committed but never acknowledged; (3) orders without items (partial writes); (4) per product, change in reserved stock = units in new orders and change in available stock = minus that (no double reservation, no lost release).

Results at a glance

#	Drill	Result	Recovery time	Durable data lost	Applied twice
D1	API process hard-killed under load	**PARTIAL**	5.9 s (5 s outage + 1.0 s start)	0 of 4,438	Risk: 2 orders committed but client got a connection error
D2a	PostgreSQL stopped 21 s	**PARTIAL**	0.7 s after DB healthy	0 of 4,439	0; but 1,284 errors returned as 500, not 503; Debezium left FAILED
D2b	PostgreSQL hung (paused) 20 s	**PASS**	0.2 s after unpause	0 of 3,429	0; hung requests cut at 10.0 s (504)
D3	Payment gateway black-holed / refusing	**FAIL**	n/a (no retry)	0	0; order row locked 10 s, 2nd click dropped at 19 s with no response
D4	Kafka broker stopped 33 s	**FAIL**	Manual: 96 s after Debezium restart	0 (all 7,868 replayed)	1,166 duplicate CDC events; Debezium stuck while reporting RUNNING
D5	CDC replay after Debezium task FAILED	**PASS**	1.8 s to RUNNING, ~60 s drain	0 of 3,299	0 duplicate events
D6	Duplicate / interrupted deliveries	**PARTIAL**	n/a	0	Webhooks: 0. POST /orders retry: duplicate order
D7	CDC worker SIGKILLed	**PASS**	83 s after container start	0 of 3,386	978 duplicate CDC events (at-least-once)
D8	Overload: DB pool 4 and 1 vs 96/300 threads	**PARTIAL**	immediate	0	0; no load shedding, queueing up to 10 s
D9	Backup taken under load, restored	**PASS**	dump 6.2 s + restore 5.5 s	0 before snapshot; 461 after (RPO)	0
-	Cache loss	N/A	-	-	No cache in the application (no Redis or in-memory cache in code or compose)

What held: order creation is one transaction, so no crash, DB stop, DB hang, broker stop or worker kill left a half-written order or drifted stock across 40,000 orders. Webhook de-duplication held under 6 concurrent copies racing 3 browser confirmations, and an interrupted webhook was rolled back and processed exactly once on redelivery. The replication slot kept every change while Debezium or Kafka was down: 0 CDC events missing in any drill.

What broke: (1) the Razorpay call ignores the request deadline and runs while holding the order row lock; (2) a client cannot safely retry POST /orders, and drills produced 16 orders whose client saw an error but which were committed; (3) Debezium does not recover by itself from a PostgreSQL restart or a Kafka outage, and in the second case its status API says RUNNING while nothing is streamed; (4) dependency outages are reported as 500 and the API never sheds load.

Findings and follow-up fixes

Ranked by impact. Each item names the drill that produced the evidence.

F1 [High] Gateway call ignores the deadline and holds the order lock (D3)

Evidence	`CreatePayment` locks the order row (`SELECT ... FOR UPDATE`) and then calls `client.Order.Create(data, nil)`, which takes no context; razorpay-go's own timeout is 10 s, equal to the API's 10 s request deadline. With the gateway black-holed: checkout 504 at 10.07 s; a cancel of the same order blocked on the lock and also failed (504, 10.02 s); a second checkout click waited for the lock, then spent another 10 s in the gateway after its own deadline and lost its connection at 19.09 s with no response (past the 15 s WriteTimeout). With the gateway refusing connections the API failed fast but returned 500 with the raw upstream error, including `https://api.razorpay.com/v1/orders` and local socket addresses.
Fix	Call the gateway outside the locked transaction (record a PENDING checkout attempt, commit, call, then attach the gateway order id); set `client.SetTimeout` well below the request deadline (e.g. 5 s); map gateway errors to 502/503 with a generic message and log the detail. Razorpay orders are safe to create twice (an orphan is never paid), so one bounded retry with jitter is acceptable there; refunds must not be retried without an idempotency key.

F2 [High] POST /orders is not idempotent; ambiguous outcomes happen (D1, D6, D7)

Evidence	Orders committed while the client saw a failure: 2 in D1 (connection reset when the process died), 6 in D7 (504 during a Docker VM I/O stall), 8 in a first D1 attempt excluded from the timings. Sending the identical request twice, even with an `Idempotency-Key` header, created 2 orders and reserved stock twice (D6-D). Any client or proxy retry duplicates the order.
Fix	Accept an `Idempotency-Key` header; store (user_id, key, request hash, response) in a table with a unique index written in the same transaction as the order; replay the stored response for repeats. The storefront should generate one key per checkout.

F3 [High] Debezium does not self-heal and can report RUNNING while stuck (D2a, D4)

Evidence	After a PostgreSQL restart the task went FAILED (*Couldn't obtain encoding for database*) and stayed FAILED until restarted by hand. After a 33 s Kafka outage the worker could not stop the old task (*failed to properly shut down, has become unresponsive*); the orphan kept walsender PID 192 on the replication slot, the new task looped on *replication slot ... is active*, and the REST status showed RUNNING throughout. Only 1,166 of 7,868 orders reached Kafka until the container was restarted. Meanwhile WAL retained by the slot grew by ~0.6 MB per 5 s under 8 threads (14 MB in total).
Fix	Alert on replication-slot lag (`pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn)`) and on CDC end-to-end lag, not on task state. Add a watchdog that restarts the task when it is FAILED and restarts the worker (or terminates the stale walsender) when lag grows while the task claims RUNNING. Set `max_slot_wal_keep_size` so a dead connector cannot fill the disk. Add `heartbeat.action.query` (writing to a small captured table): after the drills the slot showed 46.8 MB of lag while idle, because WAL from uncaptured writes (the restore drill) does not advance the slot; one order brought it to 56 bytes. Without this a lag alert fires on quiet periods.

F4 [Medium] CDC is at-least-once, the in-process consumer is at-most-once (D4, D7, code)

Evidence	Replays after a Kafka outage and a worker kill republished everything since the last offset commit (Connect commits every 60 s): 1,166 and 978 duplicate create events. The API consumer uses kafka-go `ReadMessage`, which commits the offset before the event is processed, and decode/process errors are logged and skipped with no dead-letter topic. Today the handlers only log, so nothing is harmed.
Fix	Before CDC handlers do business work: switch to `FetchMessage` + `CommitMessages` after processing, make handlers idempotent (dedupe on source LSN + table + primary key, or on a row version), and send poison messages to a DLQ topic. Add jitter to the 1 s to 30 s read backoff before running several consumers.

F5 [Medium] Dependency outages are reported as 500 (D2a, D3)

Evidence	With PostgreSQL stopped, 1,284 order requests got 500 in ~8 ms each; clients cannot tell a retryable outage from a bug. `/ready` correctly turned 503 within 0.5 s.
Fix	Map connection-refused / pool errors to 503 with `Retry-After`; keep 504 for deadline expiry. Clients should retry only 503 and only for idempotent calls (or with the F2 key).

F6 [Medium] No load shedding; readiness flaps under saturation (D8)

Evidence	Pool of 4, 96 threads: 213 orders/s, p95 1.0 s, no errors. Pool of 1, 300 threads: 93 orders/s, p50 3.1 s, p99 9.8 s, 26 x 504; 4,579 pool waits in 15 s. Nothing was rejected early: every request queued until it finished or hit the 10 s deadline. `/ready` uses the same pool with a 2 s ping and returned 503 three times.
Fix	Bound in-flight requests per instance (concurrency limiter returning 503 + Retry-After after a short queue wait, e.g. 500 ms), and decide deliberately whether readiness should fail on saturation (sheds traffic from one instance; can flap all of them at once).

F7 [Medium] Backups exist only on demand; recovery point = last dump (D9)

Evidence	A pg_dump taken under load was consistent and restorable (6.2 s + 5.5 s for 16 MB). Orders acknowledged after the snapshot (461 rows) are absent from the restore: `archive_mode=off`, nothing schedules dumps locally, and point-in-time recovery is unavailable. A restore also starts without the Debezium replication slot (not exercised).
Fix	For hosted Neon, confirm the PITR window and rehearse a branch restore; locally, schedule dumps or enable WAL archiving. Write the restore runbook including recreating the slot/publication and re-snapshotting the connector.

F8 [Low] Harness and environment notes

Evidence	On Windows a refused localhost connect takes ~2 s (SYN retries), so client-side errors during the API outage were fewer than requests attempted. Docker Desktop shows 10 to 20 s I/O stalls (also recorded in LOAD_TESTING.md); D7's API errors coincide with one and are not caused by Debezium.
Fix	Re-run D3, D4 and D7 on Linux/hosted infrastructure before drawing capacity conclusions; keep the drill harness in the repo and run it when a dependency is added.

Drill records

Each drill was scoped to the local Docker stack, time-boxed by the script (outage lengths fixed; every wait capped), and observed by the harness (per-request status and latency), the database checks above, a 0.5 s `/ready` probe where relevant, the Debezium REST status, Kafka offsets and the API/PostgreSQL/Debezium logs. Stop condition for all drills: abort and restore if a container does not become healthy within 60 s, or if any acknowledged order goes missing.

D1 API crash

Scope	API process (bin/server.exe) killed with `taskkill /F` while 16 threads placed orders; restarted 5 s later as a supervisor would.
Expected	Requests in flight fail; no partial order; stock unchanged for failed requests; service back as soon as the process is.
Recovery steps	Start the process again (no state to rebuild; migrations are idempotent).

Timeline

Time (UTC)	T+ s	Event
05:31:24.855	0.0	load started (16 threads, one-unit orders)
05:31:35.126	10.3	API process 8580 killed (taskkill /F)
05:31:40.126	15.3	restarting API
05:31:41.149	16.3	API ready again (1.02s after launch)
05:31:51.212	26.4	load stopped

Observed impact	4,438 orders acknowledged, 27 connection errors during the outage. Startup to ready: 1.0 s. 2 orders were committed whose response never reached the client.
Recovery time	5.88 s from kill to first successful order (5 s deliberate outage).
Data-loss result	0 acknowledged orders lost of 4,438; 0 orders without items; stock deltas exact.
Follow-up	F2 (idempotency key).

D2a Database unavailable (stopped)

Scope	`docker stop order-postgres` for 21 s under 8 order threads, /ready probed every 0.5 s.
Expected	Fast failure with a retryable status; /ready 503; automatic reconnect; CDC resumes.
Recovery steps	`docker start order-postgres`; the pool reconnects by itself. Debezium needed `POST /connectors/order-processing-postgres-connector/tasks/0/restart` (done in D5).

Timeline

Time (UTC)	T+ s	Event
05:32:25.407	0.0	load started (8 threads) + /ready probe every 0.5 s
05:32:37.830	12.4	docker stop order-postgres (rc=0, took 2.4s)
05:32:59.027	33.6	docker start order-postgres (rc=0, took 1.2s)
05:33:09.292	43.9	postgres container health=healthy
05:33:29.444	64.1	load stopped

Observed impact	During the outage: 1,023 x 500 (median 9 ms) and 8 x 504 (requests caught mid-shutdown, cut at 10.0 s). /ready: 503 detected after 0.52 s. Debezium task: FAILED, not restarted automatically.
Recovery time	Orders succeeded 0.69 s and /ready returned 200 1.01 s after the container was started.
Data-loss result	0 lost of 4,439; 0 unacknowledged commits; stock exact.
Follow-up	F5 (503 instead of 500), F3 (Debezium).

D2b Database slow / hung (paused)

Scope	`docker pause order-postgres` for 20 s (TCP open, no replies) under 8 order threads.
Expected	Requests bounded by the 10 s request deadline, then 504; no stuck connections afterwards.
Recovery steps	`docker unpause order-postgres`.

Timeline

Time (UTC)	T+ s	Event
05:36:25.184	0.0	load started (8 threads) + /ready probe every 0.5 s
05:36:35.339	10.2	docker pause order-postgres (rc=0, took 0.2s)
05:36:55.514	30.4	docker unpause order-postgres (rc=0, took 0.2s)
05:37:15.582	50.4	load stopped

Observed impact	9 x 504, all at 10.0 s (max 10.01 s). /ready failed after 2.17 s (2 s ping timeout). Debezium survived (RUNNING).
Recovery time	0.16 s after unpause.
Data-loss result	0 lost of 3,429; 0 unacknowledged commits; stock exact.
Follow-up	None for the API. The deadline works as designed.

D3 External dependency timeout (Razorpay)

Scope	API restarted with `HTTPS_PROXY` pointing at a local proxy that (a) accepts and never answers, (b) resets the connection. No traffic left the machine. One fresh order per mode; concurrent second checkout, cancel and read.
Expected	Checkout fails within the deadline with 502/503/504; other operations on the order are unaffected; no payment row; no hanging connection.
Recovery steps	Remove the proxy and restart the API (done at 05:38:20). Orders were left CREATED (black-hole) and CANCELLED (refuse); no payment rows.

Calls (05:37:55 to 05:38:17 UTC)

Call	Black-holed gateway	Refusing gateway
Checkout (POST /payments)	504 in 10.07 s (request timed out)	500 in 0.04 s raw upstream error in body
Second click, 1 s later	ConnectionError in 19.09 s	500 in 0.08 s raw upstream error in body
Cancel same order, 1 s later	504 in 10.02 s (request timed out)	200 in 0.2 s
GET same order	200 in 0.15 s	200 in 0.18 s

Observed impact	Black-holed: the order row stayed locked for the full 10 s, so the cancel timed out; the second click got no HTTP response (connection dropped at 19.1 s). Refusing: fast 500 that leaks the upstream URL and socket addresses. Afterwards: 0 payment rows, 0 transactions left idle-in-transaction.
Recovery time	Immediate once the gateway answers; there is no retry, so the customer must click again.
Data-loss result	No data lost or duplicated. In production the black-holed call may still have created a Razorpay order that is never used (harmless).
Follow-up	F1.

D4 Queue outage (Kafka broker)

Scope	`docker stop order-kafka` for 33 s under 8 order threads; then verify every order committed since the start appears on `order-processing.public.orders`.
Expected	API unaffected; Debezium buffers in the slot and resumes; consumer reconnects; all events delivered, duplicates possible.
Recovery steps	Kafka start (healthy after 26 s). Debezium did not resume: `docker restart order-debezium` at 05:44:17 was required.

Timeline

Time (UTC)	T+ s	Event
05:39:04.976	0.1	load started; orders topic offsets [36737, 36828, 36747]; task RUNNING
05:39:18.444	13.6	docker stop order-kafka (rc=0, 3.5s)
05:39:51.534	46.7	docker start order-kafka (rc=0, 0.7s)
05:40:17.118	72.3	kafka health=healthy
05:40:27.177	82.3	load stopped
05:40:27.681	82.8	debezium task state after recovery: RUNNING
05:43:00.026	235.2	slot backlog 13960960 bytes
05:43:23.588	258.8	API consumer lag: order_processing_cdc_consumer_lag_messages 0
05:44:17.000	-	Manual recovery: docker restart order-debezium

Time (UTC)	T+ s	Event
05:45:52.729	-	All 7,868 orders present on the topic (9,034 create events)

Observed impact	API: 3,644 orders during the outage, all 201, p95 34 ms. Consumer logged read errors and backed off. Slot backlog grew 8.2 to 11.2 MB during the outage. After Kafka returned Debezium stayed RUNNING but stuck (orphan task held the slot); 6,702 orders missing from Kafka until the manual restart.
Recovery time	API: none needed. CDC: 95.7 s after the manual restart; ~4 min of stuck CDC before that.
Data-loss result	0 orders lost in the DB; after recovery 0 missing from Kafka; 1,166 orders delivered twice.
Follow-up	F3, F4.

D5 Replay of interrupted CDC work

Scope	The Debezium task left FAILED by D2a was restarted with orders committed only in WAL since 05:32:40.
Expected	Debezium resumes from its stored LSN and publishes every missed change.
Recovery steps	`POST .../tasks/0/restart`.

Timeline

Time (UTC)	T+ s	Event
05:34:45.319	0.7	task state=FAILED, slot backlog=10257648 bytes, orders committed since 2026-10-28T05:32:40Z: 3299
05:34:45.421	0.8	POST /connectors/.../tasks/0/restart
05:34:46.454	1.8	task RUNNING
05:35:46.991	62.4	slot backlog drained to 36240 bytes
05:35:56.340	71.7	orders topic end offsets [35549, 35701, 35630] (were [34474, 34554, 34539])
05:36:11.881	87.2	read backlog: 3313 create events, 3313 distinct orders

Observed impact	3,299 orders committed while CDC was down.
Recovery time	RUNNING after 1.81 s; slot confirmed drained after ~60 s (offset flush interval).
Data-loss result	0 missing, 0 duplicates.
Follow-up	F3 (automatic restart).

D6 Duplicate and interrupted deliveries

Scope	Two PENDING payments inserted directly for fresh orders (synthetic gateway ids `order_drill...`), webhooks and checkout signatures signed with the local .env secrets; no call to Razorpay.
Expected	Each business action (capture, order PAID) applied once; failed processing leaves nothing behind and is redone on redelivery; a late failure cannot undo a capture; a retried order request does not create a second order.
Recovery steps	None needed for webhooks (gateway redelivery is the recovery). For orders: none exist; the duplicate must be cancelled by hand.

Case	What was sent	Outcome	Verdict
A	payment.captured with one event id: 6 concurrent copies + 3 concurrent browser /verify + 1 late replay	All 200. One webhook row (PROCESSED), payment SUCCESS, order PAID, stock unchanged at capture (consumed on ship)	**PASS**
B	Same event while PostgreSQL hung, then 2 redeliveries	1st: 504 in 10.01 s, nothing stored (rolled back); 2nd: 200 and processed; 3rd: 200 de-duplicated	**PASS**
C	Late payment.failed (new event id) after the capture	200; payment stays SUCCESS, order PAID	**PASS**
D	Identical POST /orders twice (client retry), with an Idempotency-Key header	2 x 201, 2 distinct orders, reserved +2	**FAIL**
Kafka	CDC replays in D4 and D7	1,166 and 978 events delivered twice	At-least-once

Data-loss result	None. Double application: none for payments; one duplicate order per retried create.
Follow-up	F2, F4.

D7 Worker crash (Debezium / Kafka Connect)

Scope	`docker kill order-debezium` (SIGKILL) under 8 order threads, started again after 29 s.
Expected	No effect on the API; on restart the connector resumes from its last committed offset; duplicates possible.

Recovery steps	`docker start order-debezium` (no manual task restart needed this time).

Timeline

Time (UTC)	T+ s	Event
05:48:35.845	0.0	load started; offsets [39690, 39899, 39765]
05:48:46.880	11.1	docker kill order-debezium (SIGKILL, rc=0)
05:49:15.524	39.7	docker start order-debezium (rc=0)
05:49:30.587	54.8	load stopped
05:50:38.408	-	All 3,386 orders present on the topic (4,364 create events)

Observed impact	API: 3,380 x 201 and 7 x 504; the 504s coincide with a Docker VM I/O stall at 05:49:00-05:49:14 (API Kafka reads to localhost timed out and PostgreSQL cancelled an inventory lock wait at the same time), not with the kill. 6 of those timed-out requests had committed.
Recovery time	82.9 s from container start to all events on the topic.
Data-loss result	0 missing from Kafka; 978 orders delivered twice; 0 acknowledged orders lost.
Follow-up	F4; F2 for the ambiguous 504s.

D8 Overload and backpressure

Scope	API restarted with `DB_MAX_OPEN_CONNS` 4, then 1 (confirmed via `go_sql_max_open_connections`), closed-loop order threads with no think time; default pool (80) restored afterwards.
Expected	Bounded latency; early rejection (429/503) once the queue is long; no corruption under contention.
Recovery steps	Remove load; restore the pool setting.

Setting	Orders/s	Outcomes	Latency p50 / p95 / p99 / max	/ready
Pool 4, 96 threads, 30 s	213.0	6,389 x 201	0.39 / 1.03 / 1.45 / 2.55 s	48 x 200
Pool 1, 300 threads, 30 s	93.4	2,802 x 201, 26 x 504	3.10 / 7.21 / 9.76 / 10.02 s	26 x 200, 3 x 503

Observed impact	Correct but unshed: requests wait in the pool queue until served or until the 10 s deadline (4,579 pool waits in 15 s at pool 1). Earlier load tests (LOAD_TESTING.md) put the real limit at commit fsync on this Docker volume, not the pool.
Data-loss result	0 lost, 0 unacknowledged commits, stock exact (2,802 orders at pool 1).
Follow-up	F6.

D9 Backup and restore

Scope	`pg_dump -Fc` of order_processing taken while 4 threads placed orders; restored with `pg_restore` into a scratch database `order_processing_restore_drill` in the same container, verified, then dropped.
Expected	Consistent snapshot; every order acknowledged before the dump is restored; restored data internally consistent.
Recovery steps	createdb + pg_restore --exit-on-error; point the API at the restored DB; recreate the Debezium slot and snapshot (not exercised).

Timeline

Time (UTC)	T+ s	Event
05:54:40.082	0.0	load started (4 threads)
05:54:45.084	5.0	pg_dump -Fc started
05:54:51.309	11.2	pg_dump finished rc=0 in 6.2s
05:54:54.380	14.3	load stopped
05:54:56.061	16.0	restore started (pg_restore into scratch DB)
05:55:01.565	21.5	pg_restore finished rc=0 in 5.5s
05:55:14.263	34.2	scratch DB dropped, dump file removed

Observed impact	Dump 16M in 6.2 s under load; restore 5.5 s. Row counts identical for users, categories, products, inventories, payments, webhooks, reviews; checksums identical for users, categories, products, payments, webhooks. Restored DB: 0 orders without items, 0 orphan items, 0 orphan payments, 0 negative stock.
Recovery time	11.7 s of database work for 16 MB (61 MB on disk).

Data-loss result	0 of 271 orders acknowledged before the dump missing. All 162 acknowledged after it absent, as are 461 rows written since the snapshot: with archive_mode=off the recovery point is the last dump.
Follow-up	F7.

Not covered, side effects, reproduction

Not covered: cache loss (no cache exists); refunds and real Razorpay retries (need a gateway sandbox or fake gateway in the running app); restoring the CDC pipeline after a database restore; several API instances or consumers; the hosted Render/Neon deployment; disk-full and clock-skew failures. Repeat the relevant drills whenever a dependency is added (Redis, a second consumer, a notification service).

Side effects on the local dev database: about 39,800 extra orders for loadtest@example.com with stock reserved on LOADTEST-0001..0010 (re-run `locust/seed.py` to reset stock), 2 synthetic payments with gateway ids `order_drill...` and 3 webhook rows. All containers were left running and healthy, the Debezium connector RUNNING and streaming, and the API running from bin/server.exe with default settings. The only repository change is this PDF (untracked, in documentation/).

Reproduction: the harness (`lib.py`, `d1_api_crash.py` ... `d9_backup_restore.py`, `verify_topic.py`) and the raw JSON results with every request record were kept in the session scratchpad (`scratchpad/drills`); copy them into the repo (e.g. `drills/`) to rerun them. Each script needs the docker-compose stack, the API on :8080 and `locust/loadtest.env`.